AI Compiler Performance Engineering

Auto-Loops Books — ch23

Contents

1 LLM and MoE Specialized Compilation **1**

1.1 LLM decode compile 3

1.2 moe scheduling 8

1.3 kv hardware 14

1.4 hetero moe 19

1.5 Key Takeaways 23

1.6 Conclusion 25

Bibliography **26**

Chapter 1

LLM and MoE Specialized Compilation

Dense decoder LLMs and mixture-of-experts (MoE) models break the assumptions generic compiler stacks were built for: batch-1 decode, growing KV residency, routed expert subgraphs, and framework-owned paging policies (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Part VI taught backend lowering; Chapter 22 taught E2E tuning loops; this chapter specializes those tools for **LLM decode compilation** and **MoE-aware schedules** across GPU, CPU, and NPU targets (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet cannot fix illegal HBM traffic when MoE routing and KV export remain unfused (Chen, 2026; NVIDIA, 2024; Liu *et al.*, 2024). Specialized compilation means every pass knows whether it is optimizing prefill, decode, or expert subgraphs—and refuses to merge them into one misleading benchmark (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020).

Four topics anchor the chapter: **LLM decode compile** (single-request residency), **MoE scheduling** (routing + expert fusion), **KV hardware adaptation** (layout and paging boundaries), and **heterogeneous MoE load balancing** (multi-SKU expert placement) (Liu *et al.*, 2024; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). DeepSeek-class deployments disaggregate prefill and decode; compiler specialization must follow that split instead of fighting it (Davies

et al., 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns: compiling whole-model graphs without decode role tags; fusing through PagedAttention handles; ignoring routing entropy in MoE profiles; promoting GPU expert schedules to CPU fallback without re-profile (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Each section below replaces one anti-pattern with a specialized compile practice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Speculative decoding and draft models add another decode subgraph: compiler specialization must treat draft and target as separate capture regions with shared KV invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Merging draft control flow into main decode capture breaks graph legality (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Quantized decode (INT4/FP8 weights, FP16 activations) changes fusion legality: specialized passes must preserve dequant residency on-chip across attention–MLP seams (SemiAnalysis, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024; AMD, 2024b). Profile bytes/token with quantization enabled—kernel names change but counters do not (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Industrial runbooks tie LLM/MoE compile artifacts to model card revisions: tokenizer, rope, expert count, and paging policy must hash into schedule fingerprints (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a;

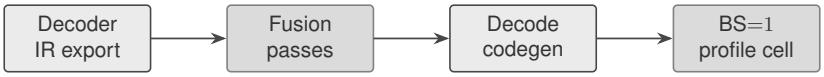


Figure 1.1: LLM decode compile path: framework export → fusion → target codegen → decode-cell validation (not prefill GEMM).

Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Silent drift between training and serving configs is a top source of "compiler regression" tickets that are actually model mismatches (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

1.1 LLM decode compile

LLM decode compile starts when the compiler owns a *decoder slice*—attention, MLP, norms, rotary, residual—exported at BS=1 with explicit KV shapes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Training-shaped graphs (large batch, no paging) mislead tile choices: shared-memory budgets that fit prefill often spill KV on decode (Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Compiler passes should tag operators with *decode role*: KV producer, KV consumer, stateless compute (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Fusion without role tags reunifies nodes that frameworks intentionally split for paging—breaking vLLM/SGLang invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

FlashAttention and Flash-Decoding shrink attention bytes but MLP/residual boundaries often remain separate kernels unless MegaKernel plans reunify them (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; NVIDIA, 2024, 2022). Decode compile therefore tracks *whole-step* bytes/token, not attention microbenchs alone (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*,

Decode compile invariant	Violation symptom
KV stays paged / blocked	capacity collapse, OOM tails
Graph capture legality	silent eager fallback
Same IR, per-SKU codegen	"works on H100" only
Bytes/token gate on merge	fast GEMM, slow decode

Table 1.1: LLM decode compile invariants for specialized compiler stacks.

2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Export boundaries matter: FX/ONNX dumps often freeze prefill shapes (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Specialized LLM pipelines inject dynamic seq-len and paging metadata before lowering (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Without metadata, "successful" compile produces undeployable schedules (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Rotary, norm, and sampling tails are easy to ignore in decode compile yet dominate launches/token on small models (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Include them in fusion plans and profile whole-step counters (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Multi-hardware delta. Hopper decode favors TMA-friendly KV tiles; XDNA requires static KV slot assignment; CPU decode fights LLC capacity—identical transformer block, different fusion limits (SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies

et al., 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 1.1.1. Decode cell (BS=1, ctx=2048): compiler fusion that materializes full KV tensors between layers doubles bytes/token even if attention kernel is optimal (Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

1.1.1 Role-tagged IR and paging contracts

Specialized decode compilation depends on role tags that generic IR does not carry (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026). Every operator in the decoder slice should be tagged as a KV producer, a KV consumer, or stateless compute, and the tags must survive lowering so backend passes can check paging legality at every seam (Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026). A fusion pass that merges across a KV boundary without understanding the framework's page contract can silently densify paged tensors, breaking capacity guarantees the serving stack relies on (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026). The paging contract should be explicit IR metadata: which tensors are paged, which block-table layout they use, and which operators may legally read through the block table (Chen, 2020; Chen and YiRage Contributors, 2026). When a pass cannot prove that a transform preserves the contract, it must refuse the transform rather than produce a correct-looking binary with broken serving semantics (Chen, 2026; Davies *et al.*, 2025).

1.1.2 Export metadata injection

Framework exports often freeze prefill-shaped graphs, so the specialized pipeline must inject dynamic-sequence and paging metadata before lowering (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2020). The injection step records context buckets, head counts, GQA and MQA structure, page sizes, and the serving framework's KV contract (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). This metadata drives tile selection, layout assignment, and fusion legality checks downstream (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). A compiler that lowers an export without this step inherits training assumptions that are false at decode time (Davies *et al.*, 2025; Kwon *et al.*, 2023). The injection should be deterministic and versioned so the same model card always produces the same decode artifact (Chen, 2020; Davies *et al.*, 2025).

1.1.3 Attention-backend selection

Attention kernels are a compiler selection problem: the same HLO or IR region can lower to a fused FlashAttention-style path, a Triton kernel, a vendor custom call, or an eager fallback (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026). Each choice has different fusion boundaries, and the selection must be made with knowledge of the surrounding MLP and norm seams (Chen, 2026; Davies *et al.*, 2025). A compiler that always selects the fastest isolated attention kernel can leave the output-projection and MLP boundaries unfused, so the decode-step counters decide rather than the attention microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026). The selection should be recorded as IR metadata so a later pass can fuse around the chosen boundary or justify why it cannot (Chen, 2020; Chen and YiRage Contributors, 2026). Attention-backend selection is where decode specialization shows up as a concrete compile-time choice (Dao *et al.*, 2022; Davies *et al.*, 2025).

1.1.4 Slice-level CI artifacts

Decode compile should be validated on slice-level CI artifacts rather than whole-model builds alone (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). The artifact set includes the exported decoder slice, the compiled per-bucket binaries, the fusion report, the byte sheet, and the decode-cell benchmark rows (Davies *et al.*, 2025; Chen, 2026). Each artifact is keyed by the model card revision so a mismatch between training and serving configurations is caught at build time (Kwon *et al.*, 2023; Chen, 2020). The slice-level gate runs on every model change, while full-model gates run on release cadence (Chen, 2026; Vellaisamy *et al.*, 2026). Slice artifacts also make cross-backend comparison tractable because the same slice can be compiled through several stacks with the same harness (Davies *et al.*, 2025; Chen, 2020).

1.1.5 Decode IR versus training IR

The same model often flows through a training-shaped export and a decode-shaped export, and the compiler should treat them as different programs (Davies *et al.*, 2025; Chen, 2020, 2026). Training IR assumes large batches, no paging, and throughput objectives; decode IR assumes batch-1, paged KV, and per-token latency (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). Sharing one artifact between the two hides the wrong trade in both directions (Chen, 2020; Davies *et al.*, 2025). The decode pipeline should therefore start from a decode-specific export or a transform that converts the training graph with explicit role

and paging metadata (Chen, 2026; Vellaisamy *et al.*, 2026). CI should gate decode artifacts on decode cells and training artifacts on training cells, never interchanging the two (Davies *et al.*, 2025; Chen, 2026).

1.1.6 Speculative and sampling tails

Speculative decoding and sampling control flow are part of the decode program even when they live outside the compiled kernel body (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The compiler should compile the draft model and the target model as separate capture regions with shared KV invariants, and the sampling logic should stay in an eager hull (NVIDIA, 2024; Chen, 2020). Merging draft branches into the target capture breaks graph legality and reintroduces host dispatch at the wrong point (Davies *et al.*, 2025; Chen, 2026). The acceptance path and the rejection path both need profiling because the target model recomputes over different KV state after rejection (Kwon *et al.*, 2023; Dao *et al.*, 2023). Compile specialization for LLMs therefore includes the scheduling shell around the kernels, not only the kernels (Chen, 2020; Vellaisamy *et al.*, 2026).

1.1.7 Numerics contracts under quantization

Quantized decode changes the numerical contract that fusion passes must preserve (SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026). Weight dequantization must happen on-chip and stay inside the fused region where possible; separate dequant kernels add both launches and bytes (Chen, 2020; Vellaisamy *et al.*, 2026). Softmax carriers and layer-norm accumulators need higher precision than the quantized matmuls, and the compiler should enforce that split by dtype metadata (Dao *et al.*, 2022; Chen, 2026). Numerical tests for quantized decode artifacts should run at the served context buckets because error grows with KV length and depth (Davies *et al.*, 2025; Dao *et al.*, 2022). The numeric contract belongs in the model card and the artifact fingerprint so a precision change is traceable (Chen, 2020, 2026).

1.1.8 Model-card artifact binding

Every decode artifact should be bound to the model card that produced it, because LLM configs drift silently between training and serving (Davies *et al.*, 2025; Chen, 2020, 2026). The binding records tokenizer version, RoPE configuration, head counts, expert topology, KV policy, and quantization format (Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). A compiler regression ticket that turns out to be a model mismatch wastes both teams' time; binding prevents it by failing

at build time when the model card changed (Chen, 2020; Davies *et al.*, 2025). The artifact manifest should hash the model card fields that affect the schedule (Chen, 2026; Vellaisamy *et al.*, 2026). Model-card binding is the specialized form of the general rule that the artifact and its configuration must be reproducible together (Davies *et al.*, 2025; Chen, 2026).

1.1.9 Decode artifact safety tests

Decode artifacts need safety tests that catch serving-contract violations before rollout (Chen, 2020, 2026; Davies *et al.*, 2025). The tests should verify paged KV stays paged through every fusion seam, graph capture remains legal at each context bucket, and no hidden allocation appears on the hot path (Kwon *et al.*, 2023; NVIDIA, 2024; Vellaisamy *et al.*, 2026). They should also check that dynamic sequence lengths select the correct bucket artifact and that fallback behavior is logged (Davies *et al.*, 2025; Chen, 2026). Safety tests run on every model-card change and every compiler change that touches KV or routing (Chen, 2020, 2026). A decode artifact that passes performance gates but fails a safety test is not deployable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.1.10 Artifact validation and signing

Decode artifacts should be validated and signed at build time so serving can verify that the loaded binary is the benchmarked artifact (Chen, 2020, 2026; Davies *et al.*, 2025). The validation step re-runs the artifact through the compile report checks and compares the fingerprint against the manifest (Chen, 2026; Vellaisamy *et al.*, 2026). Serving should refuse to load an artifact whose signature or fingerprint does not match its model card (Chen, 2020; Davies *et al.*, 2025). Signing also makes fleet forensics tractable: when a behavior changes, the loaded artifact hash identifies the exact binary and configuration (Chen, 2026; Davies *et al.*, 2025). Artifact validation and signing are the operational floor beneath all the specialization this chapter describes (Chen, 2020; Vellaisamy *et al.*, 2026).

1.2 moe scheduling

MoE scheduling is where decode compilation meets *dynamic routing* (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020). The production surprise is not expert GEMM peak FLOPs—it is launch storms when every token activates a different

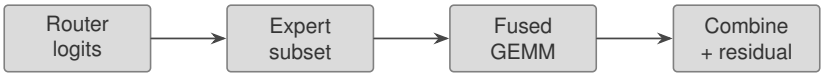


Figure 1.2: MoE scheduling: router → active experts → fused compute → combine (compile-time vs runtime split).

expert subset without batched grouping (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compiler specialization options:

1. **Compile expert templates once** per shape bucket; runtime selects indices (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

2. **Fuse router + top-k + expert prologue** when graph capture legal; isolate when control flow diverges (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

3. **Expert parallelism passes** that align with DeepEP-style dispatch when disaggregated (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

OLMoE-scale launch counts (TaxBreak) warn that MoE decode without fusion remains orchestration-bound even when experts are "fast" (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Profile launches/token *per layer* with routing entropy logged—mean ms/token hides expert imbalance tails (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. MoE compile merges require expert activation histograms from production-like traffic, not uniform random routing (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Shared-expert and routed-expert splits (DeepSeek-class) need distinct compile templates: shared experts fuse like dense MLP; routed experts batch by token groups (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Collapsing both into one schedule loses routing visibility (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

When expert counts exceed on-chip weight residency, compilation must coordinate with runtime prefetch—bytes/token includes HBM traffic for cold experts (Liu *et al.*, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b,b). MoE scheduling is a memory problem disguised as a routing problem (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

1.2.1 Routing entropy measurement

MoE compilation cannot be tuned on uniform routing, because real router distributions are skewed and bursty (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The profiling station should record per-expert activation histograms from production-like traffic and attach them to every schedule decision (Chen, 2026; Kwon *et al.*, 2023; Chen, 2020). Entropy and skew statistics belong in the MoE profile report, because they predict whether batching, prefetching, or duplication will pay (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A schedule that assumes balanced routing will thrash when real traffic concentrates on a few experts (Liu *et al.*, 2024; Chen, 2026). The histogram should be versioned with the model card and re-measured when the traffic mix changes (Chen, 2020; Davies *et al.*, 2025).

1.2.2 Expert templates and grouping

Compiling every expert as an independent kernel multiplies compile time and launch count; the specialized approach is to compile expert templates once per shape bucket and reuse them across experts of the same structure (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026). Tokens that route to the same expert can group into one dispatch when the runtime batches them, turning many small expert calls into fewer grouped kernels (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The grouping policy is a runtime decision, but the compiler must provide the templates, the buffer contracts, and the launch metadata that make grouping cheap (Chen, 2020, 2026). When expert weights differ only numerically, one compiled template serves all of them; when hidden structures differ, templates must split by structure (Liu *et al.*, 2024; Davies *et al.*, 2025). Template reuse is the compile-time half of MoE launch reduction (Chen, 2020; Vellaisamy *et al.*, 2026).

1.2.3 Router fusion legality

Fusing the router, top-k selection, and expert prologue into one captured region removes launches, but only when the control flow remains capture-safe (NVIDIA, 2024; Liu *et al.*, 2024; Davies *et al.*, 2025). Data-dependent expert counts, dynamic gather indices, and host-visible routing decisions break static capture unless the graph is bucketed (Kwon *et al.*, 2023; Chen, 2026, 2020). The legality pass should prove that the fused router has bounded top-k, static dispatch structure within a bucket, and no allocation in the captured region (NVIDIA, 2024; Chen and YiRage Contributors, 2026). When proof fails, the compiler keeps the router in an eager hull and fuses only the expert body (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Router fusion is therefore a legality decision per MoE layer, not a global feature flag (Chen, 2020, 2026).

1.2.4 Expert capacity and prefetch

When all expert weights cannot fit on-chip, the schedule must prefetch cold experts from HBM while hot experts compute (Liu *et al.*, 2024; SemiAnalysis, 2024; Chen, 2026). The prefetch distance is a compiler decision expressed as bytes moved and overlap achieved, not a runtime nicety (Chen, 2020; Davies *et al.*, 2025). The byte sheet must count cold-expert HBM traffic per token, because that traffic is part of bytes/token even when the expert kernel itself looks efficient (Chen, 2026; Vellaisamy *et al.*, 2026). Prefetch policies should follow the activation histogram:

frequently hot experts stay resident, cold experts stream (Liu *et al.*, 2024; Davies *et al.*, 2025). Compiler and runtime must agree on the resident set through metadata so a policy change does not invalidate the compiled schedule (Chen, 2020; Chen and YiRage Contributors, 2026).

1.2.5 Batched decode grouping

MoE decode at batch-1 still benefits from grouping across concurrent requests when the scheduler batches tokens through the expert layer (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025). The compiler should provide grouped dispatch templates that accept a token list per expert and a combine kernel that routes outputs back to their requests (Chen, 2020; Vellaisamy *et al.*, 2026). Grouping reduces the per-token expert launch count without changing any expert's arithmetic (Liu *et al.*, 2024; Chen, 2026). The group size and dispatch threshold are runtime parameters that should be tuned against the routing histogram, not fixed in the kernel (Davies *et al.*, 2025; Chen, 2026). Compiler and scheduler must share the same token-to-expert mapping format so grouping does not require a copy (Chen, 2020; Chen and YiRage Contributors, 2026).

1.2.6 Routing-bound classification

MoE profiles need a routing-bound classification distinct from launch, memory, and residency bounds (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A layer is routing-bound when its latency is set by expert imbalance or dispatch serialization rather than by any expert kernel (Chen, 2026, 2020). The evidence is the per-expert wait histogram: when a few experts queue while others idle, the fix is scheduling, duplication, or routing policy, not kernel tuning (Davies *et al.*, 2025; Liu *et al.*, 2024). Routing-bound labels should be recorded per layer and per bucket, because imbalance at short context may vanish at long context or vice versa (Chen, 2026; Vellaisamy *et al.*, 2026). Classifying routing separately prevents the recurring mistake of tuning expert GEMMs while the router owns the tail (Davies *et al.*, 2025; Chen, 2026).

1.2.7 Combine kernel compilation

The combine step that merges expert outputs is a separate compile surface from the expert GEMMs, and it is where routing round trips reappear if compiled carelessly (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). The combine kernel should read each selected expert's output in the layout the expert produced and accumulate

into the correct token row without an intermediate gather to dense form (Chen, 2020; Vellaisamy *et al.*, 2026). Compiler passes should fuse the combine with the following residual and norm where legal, because those seams are frequent launch sources in MoE layers (Chen, 2026; Davies *et al.*, 2025). The combine cost belongs in the MoE byte sheet, and routing-bound classifications should include it (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Combine compilation is the difference between an MoE layer that feels fused and one that merely has fast experts (Chen, 2020, 2026).

1.2.8 MoE memory planning

MoE scheduling is also a memory-planning problem because expert weights, activation buffers, and the combine workspace compete for the same on-chip and HBM budgets (Liu *et al.*, 2024; Chen, 2026; Davies *et al.*, 2025). The compiler should plan expert staging alongside activation residency so the resident-expert set and the fused region do not fight (Chen, 2020; SemiAnalysis, 2024). When weights exceed the budget, the plan must state which experts stream and what that costs per token (Chen, 2026; Vellaisamy *et al.*, 2026). Memory planning should be per-tier because on-chip capacity differs between GPU, CPU, and NPU expert hosts (AMD, 2024b,a; Goodfellow *et al.*, 2016). A schedule that ignores memory planning will produce launch-efficient MoE kernels that thrash on capacity (Liu *et al.*, 2024; Chen, 2026).

1.2.9 Per-bucket routing histograms

Routing skew is not uniform across context lengths, so histograms should be collected per bucket rather than aggregated globally (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). Long-context requests may route differently from short prompts because the attention state changes token distributions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Compile decisions that use a global histogram will be wrong for the bucket where the fleet actually spends its p99 (Chen, 2026, 2020). The artifact manifest should reference per-bucket histograms so a schedule can be justified against the bucket it serves (Liu *et al.*, 2024; Davies *et al.*, 2025). Per-bucket routing data turns MoE compilation from a load-balancing guess into an evidence-based placement (Chen, 2026; Vellaisamy *et al.*, 2026).

1.3 kv hardware

KV hardware adaptation splits responsibility: frameworks page and schedule KV blocks; compilers choose layout, fusion around KV edges, and on-chip residency for attention/MLP seams (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Illegal fusion that assumes dense KV tensors breaks PagedAttention invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Hardware-specific KV tactics:

- **GPU:** block-sparse attention tiles, TMA-friendly KV loads, graph-safe KV pointer stability (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; AMD, 2024b).

- **CPU:** cache-line blocked KV, NUMA-aware pinning for long context (Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

- **NPU/XDNA:** static KV slot tables, bounded context buckets compiled upfront (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024).

Context-length buckets matter: a schedule legal at ctx=2048 may spill at ctx=8192 when KV exceeds shared memory or LLC (Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b,b). Compile separate artifacts per bucket rather than one dynamic kernel that silently allocates (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026;

Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Multi-query and grouped-query attention change KV head layout; KV hardware passes must read head counts from model metadata, not hard-coded MHA (Kwon *et al.*, 2023; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Wrong layout passes compile yet corrupt generation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Prefix caching and long system prompts shift KV working set: compilers should not assume empty cache at step zero (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Profile cells include warm prefix lengths representative of production (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Example 1.3.1. Paged KV + fused MLP: compiler must treat KV pointers as opaque handles through attention–MLP seam—materializing paged KV to dense breaks the framework contract (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

1.3.1 KV ownership boundary

The cleanest specialization rule is that the framework owns KV block allocation and paging policy while the compiler owns layout and residency around the KV edges (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020). The IR should represent paged KV as opaque handles with a block-table contract, not as dense tensors the compiler may reshape freely (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026). Any pass that needs the dense view must insert an explicit gather whose byte cost is counted, and must prove the gather is legal under the

framework's page state (Chen, 2020; Davies *et al.*, 2025). Ownership boundaries prevent the classic failure where the compiler improves the kernel by violating the serving contract (Kwon *et al.*, 2023; Chen, 2026). The boundary also tells on-call engineers which team owns a KV regression: page capacity belongs to serving, layout and fusion belong to the compiler (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.3.2 Bucket strategy for growing KV

KV grows with context, so a single compiled artifact cannot be optimal or even legal at every length (Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026). The compiler should emit per-bucket artifacts whose on-chip staging, page walks, and fusion boundaries were chosen for that bucket's KV footprint (Dao *et al.*, 2022; Chen, 2020). Bucket boundaries should be derived from on-chip capacity thresholds, not round numbers alone, because the legal schedule changes where shared memory or LLC overflows (SemiAnalysis, 2024; Chen, 2026). The runtime selects the artifact for the current context and logs the selection so telemetry can show bucket transitions (Kwon *et al.*, 2023; Davies *et al.*, 2025). Artifacts at the longest buckets should receive extra profiling because spills concentrate there (Chen, 2026; Vellaisamy *et al.*, 2026). A bucket strategy turns KV growth from a runtime hazard into a compile-time plan (Chen, 2020; Davies *et al.*, 2025).

1.3.3 GQA, MQA, and MLA layout passes

Grouped-query and multi-head latent attention change the KV layout problem in ways a compiler must derive from model metadata rather than hard-coded assumptions (Kwon *et al.*, 2023; Dao *et al.*, 2022; Liu *et al.*, 2024). Head counts, group sizes, latent projections, and block scales determine which layout minimizes bytes and whether head-interleaving helps (Davies *et al.*, 2025; Chen, 2026, 2020). Layout passes that assume dense MHA will produce correct-looking kernels that corrupt generation for GQA or MLA models (Kwon *et al.*, 2023; Davies *et al.*, 2025). The layout derivation should be a named pass with unit tests per attention variant and per target (Chen, 2020, 2026). When a model card adds a new variant, the layout tests gate the artifact before any performance claim (Davies *et al.*, 2025; Chen, 2026).

1.3.4 KV compression and scale layout

Quantized and compressed KV caches add scale and metadata tensors whose layout must participate in the same passes as the cache itself (Liu *et al.*, 2024; Davies *et al.*, 2025; SemiAnalysis, 2024). Per-block scales, group offsets, and latent projections change the byte sheet and the legal fusion boundaries around attention (Chen, 2026; Dao *et al.*, 2022). The compiler must keep dequant and scale logic inside the fused attention region where possible, and must count any separate scale pass as bytes moved (Chen, 2020; Davies *et al.*, 2025). Precision rules apply: softmax and accumulator paths need higher precision even when KV bytes are reduced (Dao *et al.*, 2022; Chen, 2026). KV compression is a residency and bytes optimization that only pays when its layout and numerics are compiled together with the attention kernel (Liu *et al.*, 2024; Chen, 2026).

1.3.5 Write and read asymmetry

KV cache hardware adaptation must respect the asymmetry between writes and reads (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026). Each new token writes one KV pair, then every subsequent token reads the full prefix, so layouts that optimize read bandwidth may pay for writes once and layouts that optimize writes may tax every read (Dao *et al.*, 2023; Chen, 2026). The compiler should make the trade explicit by counting both directions in the byte sheet (Chen, 2020; Vellaisamy *et al.*, 2026). At long context, read bandwidth dominates and attention tile shapes should follow the read pattern; at short context, write cost is proportionally larger (Davies *et al.*, 2025; Chen, 2026). A per-bucket layout decision that tracks this asymmetry is the KV analog of the prefill-decode split (Kwon *et al.*, 2023; Chen, 2026).

1.3.6 Prefix-aware compile cells

Production KV caches are rarely empty at step zero, so compile and profile cells must include warm prefixes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The warm-prefix length should match the product's typical prompt distribution, because attention over a long system prompt has different residency and latency behavior than a cold start (Chen, 2026; Davies *et al.*, 2025). Cells that only measure empty caches optimize the wrong operating point and miss prefix-cache interactions (Chen, 2020, 2026). The compile cell matrix should therefore list cold, short-prefix, and long-prefix rows with the same counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Prefix-aware cells are how decode compilation survives

real serving workloads (Kwon *et al.*, 2023; Chen, 2026).

1.3.7 Attention input tiling

KV layout and attention tiling must be chosen together because the tile shape determines how block-table pages map onto on-chip memory (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen, 2026). A tile that spans several physical pages changes the page-walk cost; a tile aligned to page boundaries keeps the walk regular (Kwon *et al.*, 2023; Chen, 2026). The compiler should derive tile dimensions from the page size, head dimension, and on-chip budget, and should expose the derivation in the compile report (Chen, 2020; Davies *et al.*, 2025). When the framework changes page size, the tiles must be re-derived and the artifact re-gated (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Attention input tiling is where KV hardware adaptation becomes a concrete compiler decision with measurable byte impact (Chen, 2026; Dao *et al.*, 2022).

1.3.8 Graph-safe KV pointer handling

Captured decode graphs need KV pointers that are stable across replay or a graph-update mechanism that the compiler and runtime both understand (NVIDIA, 2024; Kwon *et al.*, 2023; Chen, 2026). The compiler should distinguish buffers that are fixed for the capture lifetime from buffers that advance per token, and generate the update path accordingly (Davies *et al.*, 2025; Chen, 2020). Paged KV adds a second indirection: the page table itself may move as blocks allocate, so the capture contract must pin or update both the data pointers and the table (Kwon *et al.*, 2023; Chen, 2026). A graph that silently falls back to eager when KV state changes is a p99 regression hiding in plain sight (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Graph-safe KV pointer handling is a compile-time and runtime contract that must be tested together (NVIDIA, 2024; Chen, 2026).

1.3.9 Page-size metadata contract

The framework's page size is an input to compiler layout, so the artifact should record it as part of the model card and reject mismatches at load (Kwon *et al.*, 2023; Chen, 2020, 2026). Changing page size changes tile alignment, block-walk costs, and bucket legality, so serving and compiler must agree on the value through the artifact manifest (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A mismatch between the compiled page assumption and the runtime policy is a correctness and performance hazard that only manifests at long context (Chen, 2026; Davies

et al., 2025). The page-size contract is the smallest concrete example of the KV ownership boundary in action (Kwon *et al.*, 2023; Chen, 2020). It is enforced by the same manifest hash that guards every other decode artifact, which keeps the framework and compiler from drifting apart without a failing build, and it makes page-size drift a build failure rather than a long-context incident (Chen, 2020, 2026).

1.4 hetero moe

Heterogeneous MoE load balancing spans *compile-time* expert placement and *runtime* routing policy (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Expert parallelism across GPU tiers (prefill pool vs decode pool) fails when compiled expert weights assume uniform NVLink topology (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compiler stacks should emit *expert placement metadata*: which experts are colocated, which tiers use INT4/FP8 kernels, and which backends are legal per chip (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Runtime load balancers (eplb-class policies in open DeepSeek infra) consume that metadata; compilers should not hard-code routing (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

On CPU/NPU fallback paths, subset experts may run on host while GPU serves hot experts—compilation must quarantine backends per expert, not share one schedule (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hetero MoE gates in Chapter 22 apply per tier: bytes/token and launches/token on each participating SKU (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon

et al., 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Fleet skew—one expert dominating activations—is a *routing* problem first, but compilation can mitigate via duplicated hot-expert kernels on multiple chips when policy allows (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Measure load imbalance with production histograms; synthetic uniform routing hides tail latency (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 24 extends heterogeneity to cluster deployment; Chapter 28 clarifies framework vs compiler KV ownership (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Specialized LLM/MoE compilation succeeds when it respects those boundaries (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.4.1 Expert placement metadata schema

Heterogeneous MoE compilation should emit placement metadata that the runtime consumes rather than embedding routing policy in compiled kernels (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020). The schema records expert identity, target chip, precision, backend legality, co-residency constraints, and the byte cost of moving that expert to another chip (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2026). Load balancers use this metadata to route tokens while the compiler uses it to pick schedules (Liu *et al.*, 2024; Davies *et al.*, 2025). The schema must be versioned with the model card so a placement change is traceable (Chen, 2020; Vellaisamy *et al.*, 2026). When compiler and runtime disagree about where an expert runs, the metadata is the single source of truth for the postmortem (Davies *et al.*, 2025; Chen, 2026).

1.4.2 Per-tier fallback quarantine

Mixed hardware deployments need per-tier quarantine so a backend that cannot run an expert never receives it silently (AMD, 2024b,a; Chen, 2020). The compiler should emit a legality list per expert and per backend, and the runtime should consult it before dispatch (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026). A fallback to CPU or NPU is acceptable only when it is intentional, logged, and measured with that tier's counters (Goodfellow *et al.*, 2016; AMD, 2024a; Vellaisamy *et al.*, 2026). Silent fallback is how heterogeneous fleets discover that a GPU-only schedule was shipped to an NPU cell (AMD, 2024b; Davies *et al.*, 2025). Quarantine lists turn backend mismatch from a runtime surprise into a compile-time and dispatch-time check (Chen, 2020, 2026).

1.4.3 Skew mitigation through duplication

When production histograms show a few experts dominating activation, duplication can reduce the routing tail (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The compiler's role is to compile duplicated copies of hot experts for the chips that will host them and to record the duplication in placement metadata (Chen, 2020, 2026). Duplication is only worthwhile when the load imbalance exceeds the memory cost of the extra copies (Davies *et al.*, 2025; Chen, 2026). The decision should be revisited when histograms change, because a traffic shift can make yesterday's duplication wasteful (Liu *et al.*, 2024; Chen, 2020). Skew mitigation is a joint routing and compilation decision, and the compiler half is providing cheap, correct duplicate artifacts (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.4.4 Fleet-level gates

Heterogeneous MoE changes must pass gates on every participating tier, not only on the tier where the change was developed (Davies *et al.*, 2025; AMD, 2024b; Chen, 2026). The gate matrix from Chapter 22 extends with columns for expert dispatch cost, load imbalance, and fallback rate per tier (Chen, 2020; Vellaisamy *et al.*, 2026). A placement change that improves the GPU tier but raises CPU fallback should fail unless the fallback is intentional and within budget (AMD, 2024a; Davies *et al.*, 2025). Fleet gates also verify that the placement metadata the runtime consumes matches the artifacts the compiler emitted (Chen, 2020, 2026). Cross-tier gate reports should be published with the same schema as single-tier reports so comparisons stay readable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.4.5 Placement simulation

Expert placement across heterogeneous tiers should be simulated before it is deployed, because the routing distribution and the network topology interact in ways intuition misses (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). The simulation replays production routing histograms over candidate placements and reports per-tier expert latency, cross-tier transfer bytes, and imbalance (Chen, 2020; Vellaisamy *et al.*, 2026). Compilation supplies the byte and kernel costs per candidate; the simulator combines them with routing (Liu *et al.*, 2024; Davies *et al.*, 2025). A placement that looks balanced on average can still produce tails when a hot expert sits on a slow tier (Chen, 2026; Davies *et al.*, 2025). Placement simulation is the cheapest place to test duplication, fallback, and load-balancer policies before fleet rollout (Chen, 2020; Vellaisamy *et al.*, 2026).

1.4.6 Topology-aware metadata

Placement metadata must include topology facts: which chips share NVLink or coherent memory, which pairs pay PCIe or network transfer costs, and which tiers are in different failure domains (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020). Expert movement costs depend on these facts, and the compiler cannot choose sane schedules without them (Chen, 2026; Vellaisamy *et al.*, 2026). Topology metadata belongs in the artifact manifest and should be regenerated when the fleet layout changes (Davies *et al.*, 2025; Chen, 2026). A placement computed for one cluster shape silently degrades when the cluster is rebuilt without the metadata update (Chen, 2020; Davies *et al.*, 2025). Topology-aware metadata is what makes heterogeneous MoE compilation reproducible across fleet changes (Liu *et al.*, 2024; Chen, 2026).

1.4.7 Placement rollback

Heterogeneous placement changes need the same rollback discipline as kernel changes (Davies *et al.*, 2025; Chen, 2026, 2020). The previous placement metadata and the artifacts it references must be retained so a load-imbalance regression can be reverted in one dispatch-policy update (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Rollback should be testable at the dispatch layer without redeploying expert kernels, because the placement metadata and the kernels are separate artifacts (Chen, 2020; Davies *et al.*, 2025). When a rollback occurs, the postmortem compares the placement metadata diff to explain the imbalance shift (Liu *et al.*, 2024; Chen, 2026). Placement rollback turns heterogeneous MoE tuning into an

iterative operation instead of a risky one-way change (Davies *et al.*, 2025; Chen, 2026).

1.4.8 Telemetry mapping for placement

Heterogeneous MoE telemetry should map every request to the placement metadata version and the dispatch decision that served it (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). That mapping lets a load-imbalance regression be traced to a routing-policy change, a placement change, or a traffic shift (Chen, 2020; Vellaisamy *et al.*, 2026). Per-expert latency and transfer bytes should be tagged with the expert's target tier and fallback status (Davies *et al.*, 2025; Chen, 2026). When the fleet scales, the telemetry reveals whether the placement still matches the topology (Liu *et al.*, 2024; Chen, 2026). Placement telemetry turns heterogeneous MoE from a compile-time bet into a measured, maintainable system (Chen, 2020; Davies *et al.*, 2025).

1.5 Key Takeaways

- **Own the decoder slice at batch-1.** LLM decode compilation begins when the compiler owns a whole decoder slice—attention, MLP, norms, rotary, residual—exported at batch-1 with explicit KV shapes, and gates it on whole-step bytes/token rather than per-operator FLOPs (Kwon *et al.*, 2023; Davies *et al.*, 2025). Optimizing the slice as a unit while respecting the framework's paging is what keeps the attention-MLP seam resident (Dao *et al.*, 2022; Chen, 2026).

- **MoE compilation meets dynamic routing.** The hard part of MoE scheduling is data-dependent routing: template the experts so they share a kernel, fuse the router prologue only when legal, and profile launches/token under realistic routing entropy rather than a uniform-load proxy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). A schedule that looks fine on balanced routing can thrash when real traffic skews load across experts (Davies *et al.*, 2025; Kwon *et al.*, 2023).

- **KV is a split responsibility.** Frameworks page and schedule KV blocks; the compiler chooses layout, fuses around the KV edges, and secures on-chip residency for the attention/MLP seam—and that split is realized differently per GPU, CPU, and NPU (Kwon *et al.*, 2023; AMD, 2024b). Treating KV layout as a compiler transform rather than a runtime afterthought is what

makes the paged attention fast instead of merely correct (Dao *et al.*, 2022; SemiAnalysis, 2024).

- **Heterogeneous MoE spans compile-time and runtime.** Load balancing splits across compile-time expert placement and runtime routing policy, so the compiler emits placement metadata and per-tier gates while the runtime routes, and backends are quarantined across fallback paths (Liu *et al.*, 2024; Davies *et al.*, 2025). Without per-tier gates a placement tuned for one accelerator silently regresses another (Chen and YiRage Contributors, 2026; AMD, 2024a).
- **MoE and KV specialization still defer to the decode counters.** However specialized the compilation, the acceptance metric is unchanged: bytes/token and launches/token at the batch-1 decode operating point, not peak FLOPs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). MoE's launch density and KV's bandwidth pressure are precisely the terms those counters expose (Chen, 2026; Goodfellow *et al.*, 2016).

Practical decision rules. Six rules summarize LLM and MoE specialization for a working compiler team. First, tag every decode operator with a KV or routing role and keep those tags through lowering, because untagged fusion breaks serving contracts (Kwon *et al.*, 2023; Chen, 2020). Second, inject dynamic-sequence, paging, and head metadata before lowering and bind the artifact to the model card (Davies *et al.*, 2025; Chen, 2026). Third, compile expert templates once per shape bucket and group tokens at runtime, because launch reduction in MoE comes from grouping, not from faster single experts (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Fourth, treat KV layout, bucket strategy, and attention tiling as compile-time decisions derived from the framework's page contract (Kwon *et al.*, 2023; Chen, 2026). Fifth, emit placement metadata and topology facts for heterogeneous fleets and let the runtime route against them (Liu *et al.*, 2024; Davies *et al.*, 2025). Sixth, gate every artifact on whole-step bytes/token and launches/token at the served buckets with realistic routing and warm prefixes (Chen, 2026; Davies *et al.*, 2025). Teams that apply these rules get specialized compilation that survives serving reality; teams that skip them get kernels that are fast in the lab and broken in the fleet (Vellaisamy *et al.*, 2026; Chen, 2026).

1.6 Conclusion

LLM and MoE specialized compilation narrows generic compiler stacks to decode-shaped graphs, routed experts, and KV contracts frameworks own (Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Gate merges on bytes/token, launches/token, and paging legality—not expert GEMM peaks alone (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Chapter 24 carries heterogeneity to cluster scale; Chapter 25 automates search inside the same invariants (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 3, 4, 5, 9, 10, 13, 14, 15, 19, 20, 21, 24, 25

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 9, 10, 13, 14, 15, 19, 20, 21, 23, 25

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 14, 15, 19, 20, 21, 24, 25

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 2, 3, 4, 5, 6, 7, 9, 10, 14, 15, 17, 19, 20, 25

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 9, 10, 14, 15, 16, 17, 18, 19, 20, 23, 24, 25

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 9, 10, 13, 14, 15, 19, 20, 21, 24, 25

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14, 15, 16, 17, 18, 19, 20, 23, 24, 25

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 3, 4, 5, 9, 10, 14, 15, 19, 20, 25

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 14, 15, 18, 19, 20, 25

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 7, 9, 10, 11, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25